DSA 8020 Project III

Megan Collins

2026-04-26

```
library(fields)
```

```
## Loading required package: spam
```

```
## Warning: package 'spam' was built under R version 4.5.2
```

```
## Spam version 2.11-3 (2026-01-05) is loaded.
## Type 'help( Spam)' or 'demo( spam)' for a short introduction
## and overview of this package.
## Help for individual functions is also obtained by adding the
## suffix '.spam' to the function name, e.g. 'help( chol.spam)'.
```

```
##
## Attaching package: 'spam'
```

```
## The following objects are masked from 'package:base':
##
##     backsolve, forwardsolve
```

```
## Loading required package: viridisLite
```

```
## Loading required package: RColorBrewer
```

```
##
## Try help(fields) to get started.
```

```
library(maps)
```

Part I: Time Series Analysis

Load and prepare the data

```
load("/Users/mrcolli/Desktop/DSA 8020/NA_MaxT_ERA.RData")

NA_MaxT <- NA_MaxT - 273.15
```

```r
nlon <- length(NA_lon)
nlat <- length(NA_lat)
nmon <- 12
nyr <- 39

ls()
```

```
##  [1] "day"     "mon"     "NA_lat"  "NA_lon"  "NA_MaxT" "nlat"    "nlon"
##  [8] "nmon"    "nyr"     "yr"
```

The temperature data were loaded from the NA_MaxT_ERA.RData file. Since the original values were recorded in Kelvin, they were converted to Celsius by subtracting 273.15 to improve interpretability. Key dimensions of the dataset, including the number of longitude points, latitude points, months, and years, were identified to better understand the structure of the data and support subsequent time series analysis.

Select a location

```r
Clemson.lon.lat <- c(-82.8374, 34.6834)

dist2Clemson <- rdist.earth(
  matrix(Clemson.lon.lat, 1, 2),
  expand.grid(NA_lon, NA_lat),
  miles = FALSE
)

id <- which.min(dist2Clemson)

lon_id <- id %% nlon
lat_id <- id %/% nlon + 1

lon_id
```

```
## [1] 58
```

```r
lat_id
```

```
## [1] 15
```

Clemson, South Carolina was used for the location for this analysis. Since the temperature data is stored on a spatial grid, the exact location may not be available in the dataset. Therefore, I calculated the distance from Clemson to each grid point and selected the nearest grid point.

Part I-A: Yearly Time Series

```r
ts_daily <- NA_MaxT[lon_id, lat_id, ]

ts_yearly <- tapply(ts_daily, list(yr), mean)
```

The original data contain daily maximum temperatures. I converted the daily data into yearly averages by taking the mean temperature for each year. This transformation is necessary because yearly averages reduce daily noise and make it easier to study long-term temperature trends.

Plot Time Series

```
time_yearly <- as.numeric(names(ts_yearly))

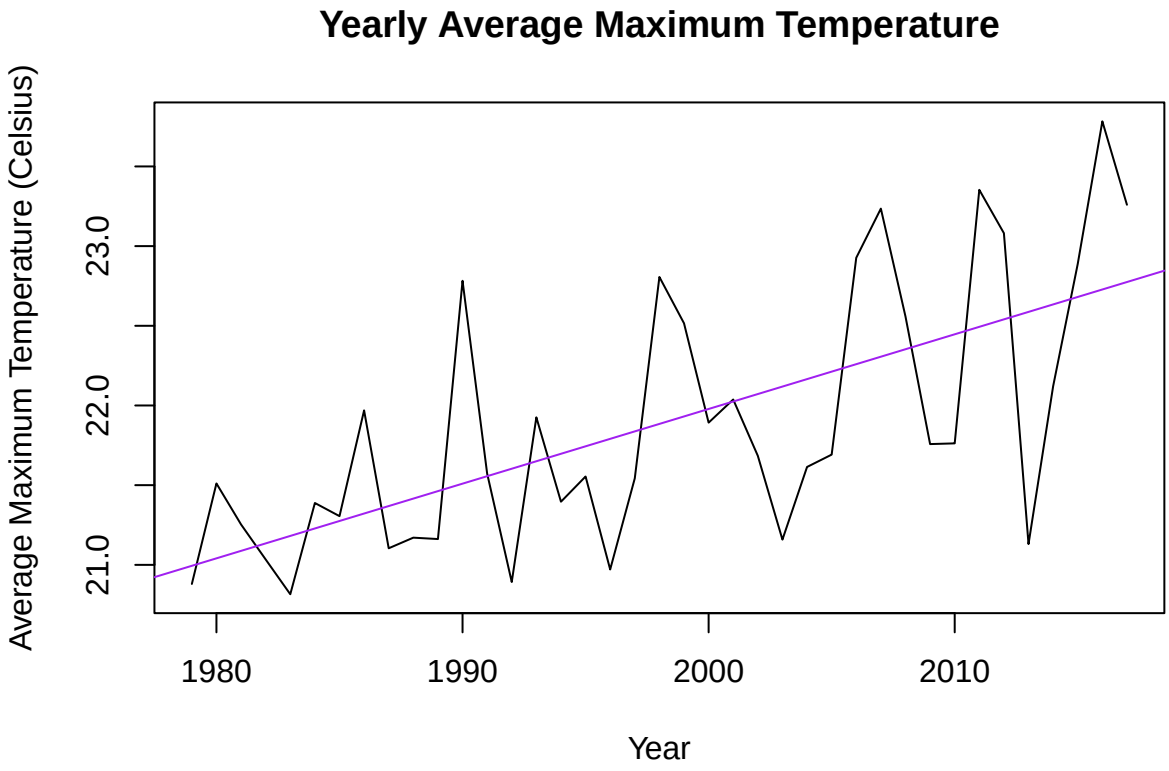

plot(time_yearly, ts_yearly,
     type = "l",
     xlab = "Year",
     ylab = "Average Maximum Temperature (Celsius)",
     main = "Yearly Average Maximum Temperature")

lm_yearly <- lm(ts_yearly ~ time_yearly)

summary(lm_yearly)
```

```
##
## Call:
## lm(formula = ts_yearly ~ time_yearly)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.45689 -0.37676 -0.08577  0.47737  1.27304
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -71.733892  17.515576  -4.095  0.00022 ***
## time_yearly   0.046856   0.008766   5.345 4.84e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6161 on 37 degrees of freedom
## Multiple R-squared:  0.4357, Adjusted R-squared:  0.4205
## F-statistic: 28.57 on 1 and 37 DF,  p-value: 4.839e-06
```

```
abline(lm_yearly, col = "purple")
```

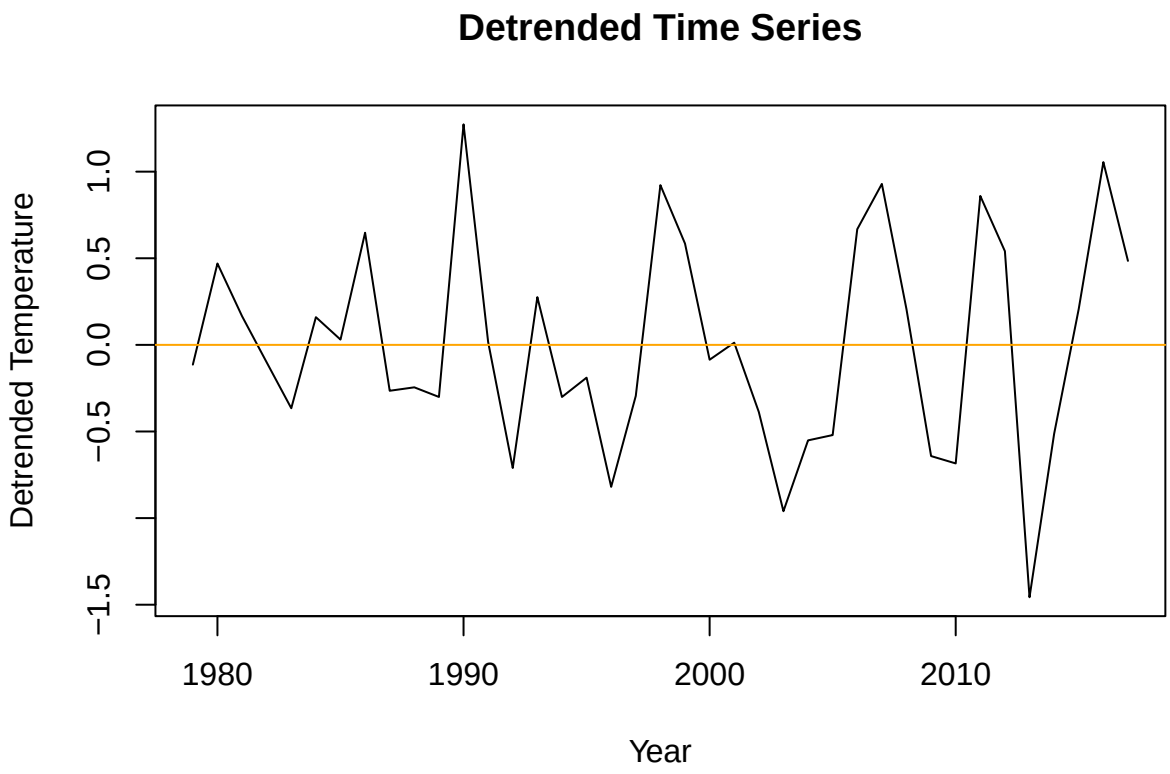


The time series plot shows the yearly average maximum temperature over time, with noticeable year-to-year variability. The fitted linear regression indicates a statistically significant upward trend (p-value < 0.001), with temperatures increasing at an estimated rate of approximately 0.047°C per year. The model explains about 43.6% of the variability in the data ($R^2 = 0.4357$), suggesting a moderate fit. While short-term fluctuations are present, there are no extreme outliers, and the variability appears relatively stable over time. Overall, this indicates a gradual increase in average maximum temperatures over the study period.

Linear Trend

```
ts_yearly_detrended <- residuals(lm_yearly)

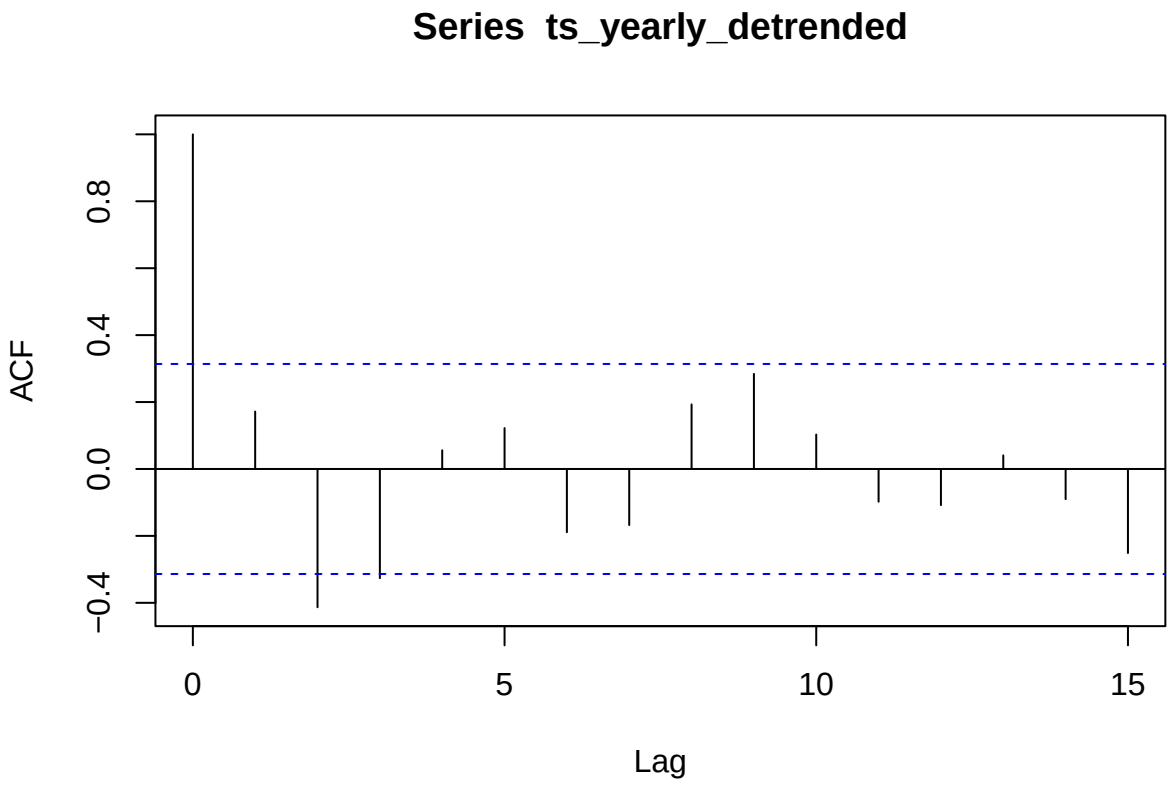
plot(time_yearly, ts_yearly_detrended,
     type = "l",
     xlab = "Year",
     ylab = "Detrended Temperature",
     main = "Detrended Time Series")
abline(h = 0, col = "orange")
```



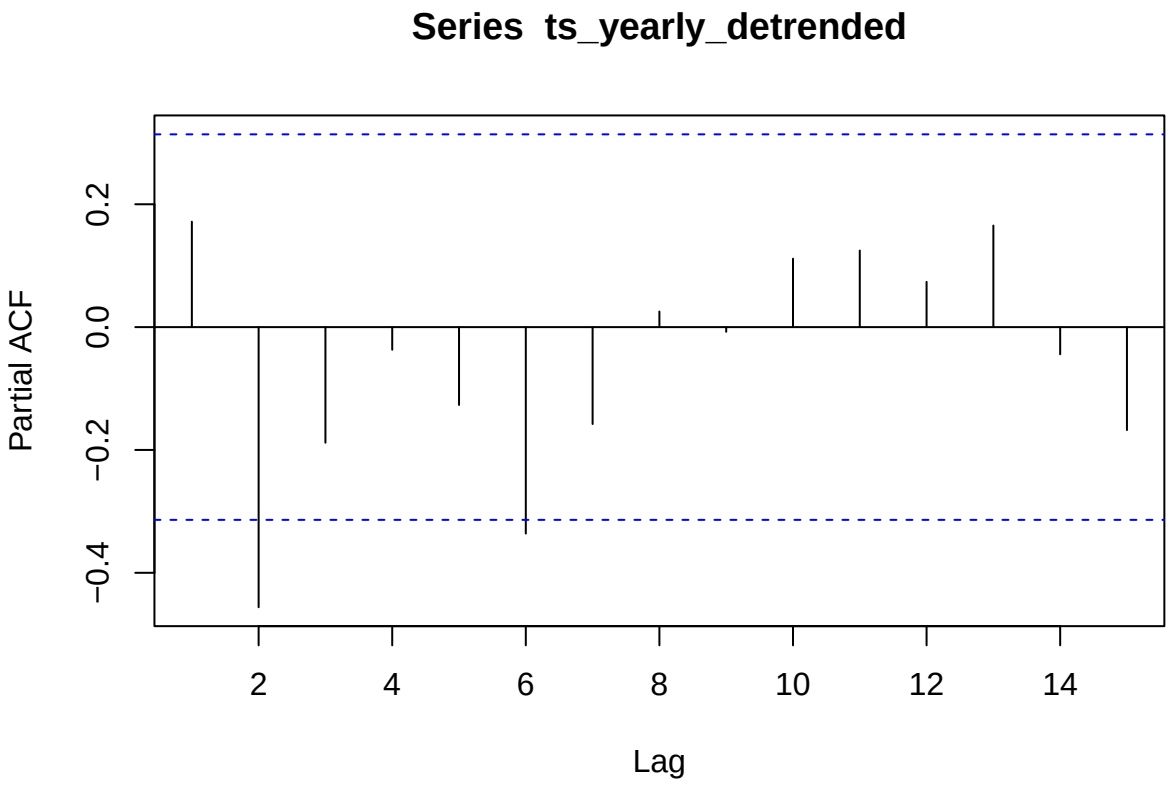
The detrended time series represents the residuals after removing the linear trend from the original data. The plot shows that the values fluctuate around zero, indicating that the long-term trend has been effectively removed. The remaining variation reflects short-term fluctuations in temperature, with no clear systematic pattern over time. The variability appears relatively constant, supporting the assumption of stationarity. This is important because many time series models, such as ARMA, require a stationary series to produce reliable results.

ACF and PACF

```
acf(ts_yearly_detrended)
```




```
pacf(ts_yearly_detrended)
```



The ACF and PACF plots were used to identify appropriate ARMA models for the detrended time series. The ACF shows a significant spike at lag 1 followed by a rapid decay, which is consistent with a short-memory process. The PACF displays a notable spike at lag 2, after which the values fall within the significance bounds. This pattern suggests that an autoregressive model of order 2 (AR(2)) may be appropriate. However, these plots provide only initial guidance, and candidate models must be further evaluated using model selection criteria such as AIC and BIC.

Candidate ARMA Models

```
fit1 <- arima(ts_yearly_detrended, order = c(1, 0, 0))
fit2 <- arima(ts_yearly_detrended, order = c(0, 0, 1))
fit3 <- arima(ts_yearly_detrended, order = c(1, 0, 1))

AIC(fit1, fit2, fit3)
```

```
##      df      AIC
## fit1  3 75.69328
## fit2  3 73.63843
## fit3  4 74.86666
```

```
BIC(fit1, fit2, fit3)
```

```
##      df      BIC
## fit1  3 80.68397
```

```
## fit2  3 78.62911
## fit3  4 81.52090
```

Three candidate ARMA models were fit to the detrended time series: AR(1), MA(1), and ARMA(1,1). Model performance was evaluated using AIC and BIC. The MA(1) model produced the lowest AIC (73.64) and BIC (78.63), indicating the best tradeoff between model fit and complexity among the candidates. Although the ACF and PACF initially suggested an autoregressive structure, the information criteria favor the MA(1) model. Therefore, the MA(1) model was selected as the most appropriate model for the detrended series.

Forecast

```
forecast <- predict(fit2, n.ahead = 1)

pred <- forecast$pred
se <- forecast$se

lower <- pred - 1.96 * se
upper <- pred + 1.96 * se

lower
```

```
## Time Series:
## Start = 40
## End = 40
## Frequency = 1
##      pred
## -1.083696
```

```
upper
```

```
## Time Series:
## Start = 40
## End = 40
## Frequency = 1
##    pred
## 1.16961
```

Using the selected MA(1) model, the one-step-ahead forecast for the detrended yearly temperature series is 0.043, with a standard error of 0.575.

A 95% prediction interval is given by:

$$0.043 \pm 1.96 \times 0.575$$

which results in an interval of approximately:

$$(-1.084,\ 1.170).$$

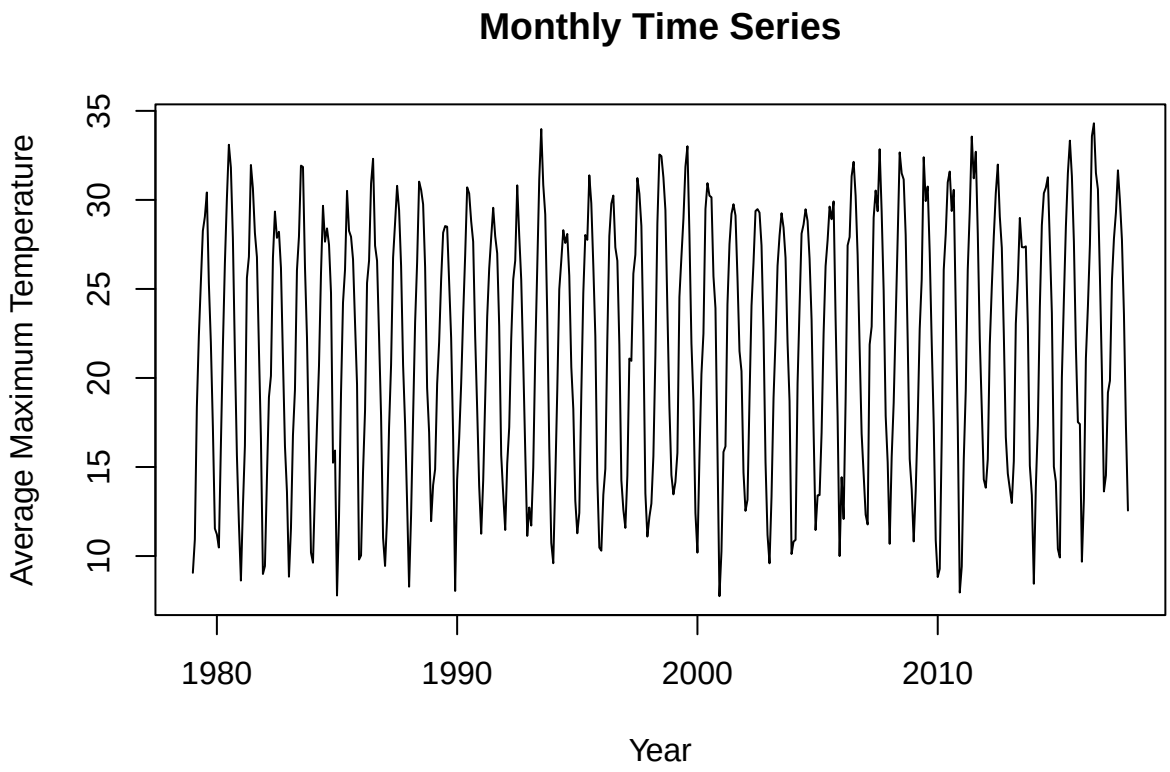
The relatively wide prediction interval indicates a moderate level of uncertainty in the forecast, reflecting the variability remaining in the detrended series. This suggests that while the model captures short-term dependence, there is still inherent randomness that limits precise prediction.

Part I-B: Monthly Time Series

```
ts_monthly <- tapply(ts_daily, list(mon, yr), mean)

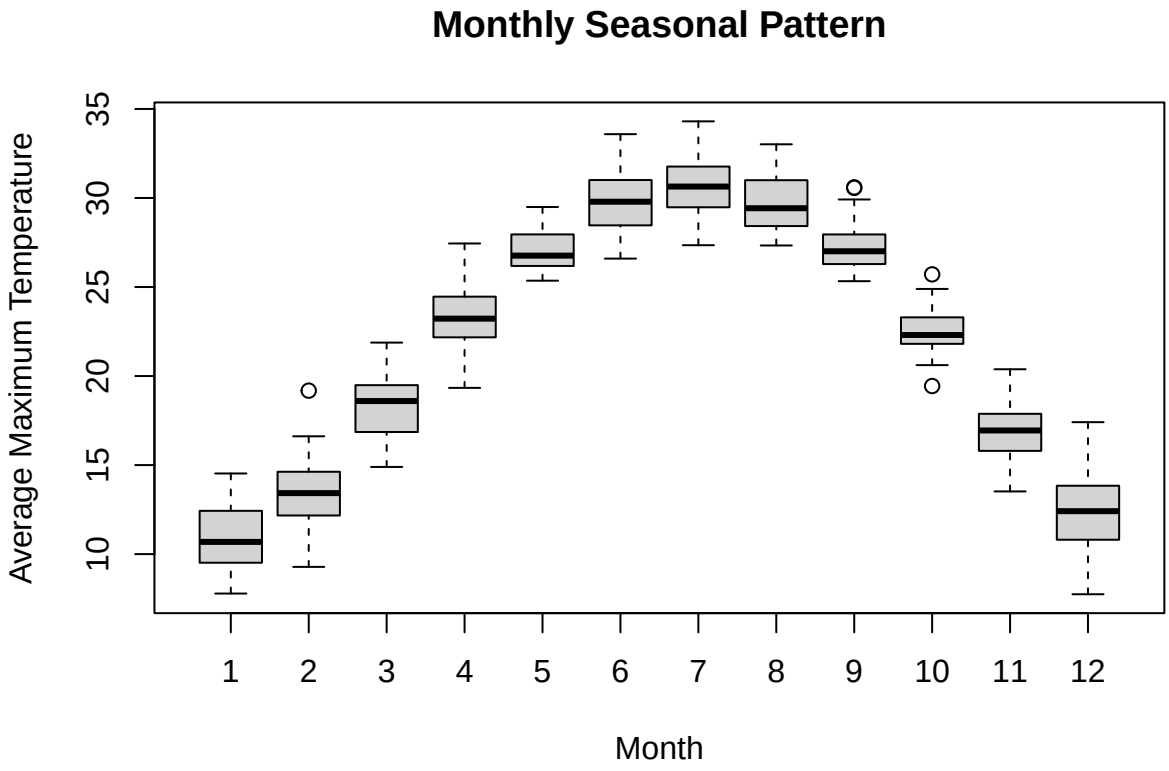
monthly_ts <- ts(as.vector(ts_monthly),
                 frequency = 12,
                 start = c(1979, 1))

plot(monthly_ts,
     xlab = "Year",
     ylab = "Average Maximum Temperature",
     main = "Monthly Time Series")
```



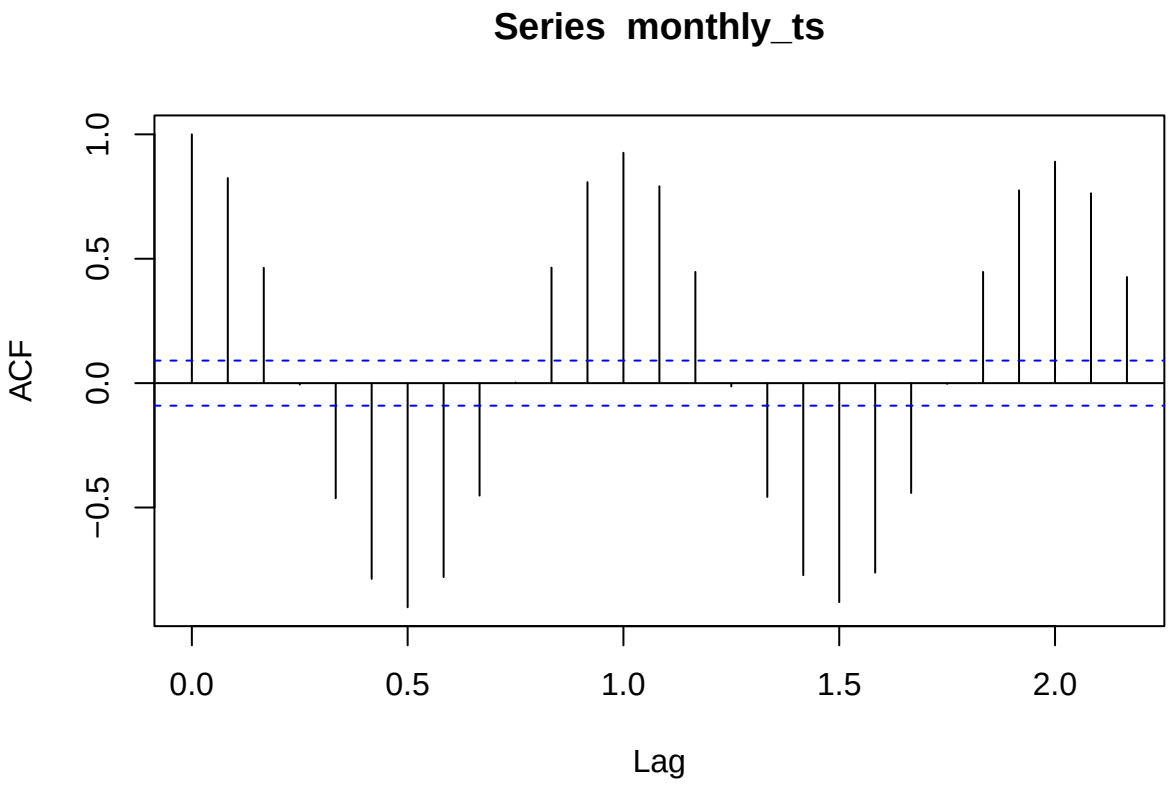
The daily data were aggregated into monthly averages by computing the mean temperature for each month and year. The resulting time series exhibits a strong and consistent seasonal pattern, with temperatures rising and falling in a repeating annual cycle. Peaks occur during the warmer months, while lower values occur during the colder months, indicating clear seasonality with a period of 12 months. Although there is no pronounced long-term trend, the dominant seasonal structure suggests that variability in the data is primarily driven by recurring annual patterns rather than long-term changes.

```
boxplot(as.numeric(monthly_ts) ~ cycle(monthly_ts),
        xlab = "Month",
        ylab = "Average Maximum Temperature",
        main = "Monthly Seasonal Pattern")
```

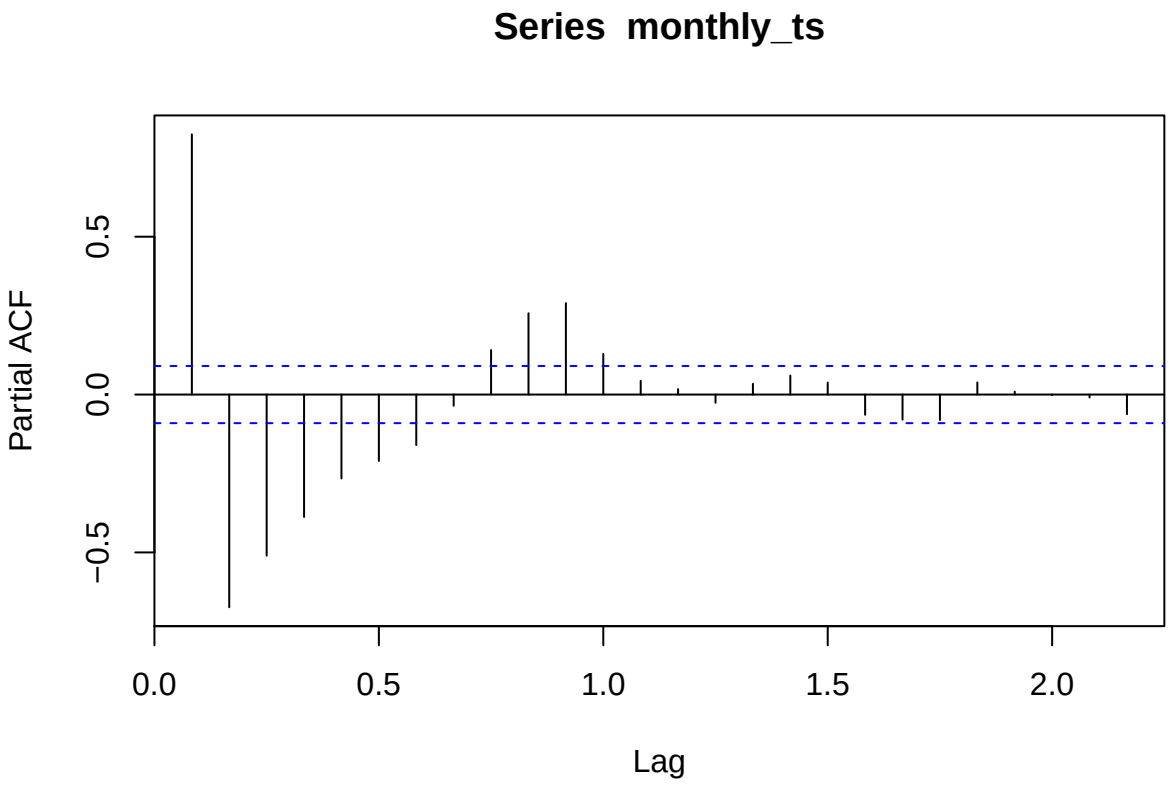


The boxplot shows the distribution of monthly average maximum temperatures across all years. Temperatures are lowest in the winter months, especially January and December, and highest in the summer months, especially June through August. The gradual increase from winter to summer and decrease from summer to winter confirms a strong annual seasonal pattern. This supports the conclusion that month is an important driver of temperature variation and should be included when modeling the monthly time series.

```
acf(monthly_ts)
```




```
pacf(monthly_ts)
```




```
fit_monthly <- arima(monthly_ts,
                     order = c(1, 0, 1),
                     seasonal = list(order = c(1, 0, 1),
                                     period = 12))

fit_monthly
```

```
##
## Call:
## arima(x = monthly_ts, order = c(1, 0, 1), seasonal = list(order = c(1, 0, 1),
##     period = 12))
##
## Coefficients:
##          ar1      ma1    sar1     sma1  intercept
##       0.6122  -0.3578  0.9998  -0.9306    21.8681
## s.e.  0.1293   0.1522  0.0002   0.0292     2.6412
##
## sigma^2 estimated as 2.745:  log likelihood = -926.96,  aic = 1865.91
```

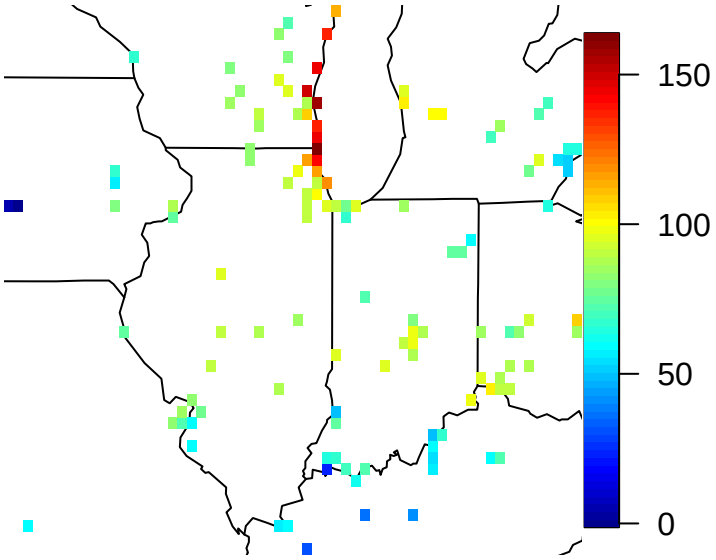
The ACF and PACF plots were used to examine the dependence structure of the monthly time series. The ACF displays a clear repeating pattern with significant spikes at seasonal lags (multiples of 12), indicating strong annual seasonality. The PACF shows notable correlations at lower lags, suggesting the presence of short-term dependence in addition to the seasonal pattern. Together, these results indicate that the series exhibits both short-term and seasonal dependence, which must be accounted for in the model. Therefore, a seasonal ARIMA (SARIMA) model is appropriate for capturing both the temporal and seasonal dynamics of the monthly time series.

Part II: Spatial Interpolation

```
data(ozone2)

loc <- ozone2$lon.lat
rg <- apply(loc, 2, range)
y <- ozone2$y[16, ]
good <- !is.na(y)

map("state", xlim = rg[,1], ylim = rg[,2])
quilt.plot(loc[good, ], y[good], nx = 60, ny = 48, add = TRUE)
```



The ozone dataset was used to examine spatial variation in ozone concentrations across locations. Longitude and latitude coordinates were extracted along with ozone measurements for a selected time point, and missing values were removed to ensure valid observations. A map was first generated to provide geographic context, and the quilt plot was used to visualize ozone concentrations, where color intensity represents the magnitude of ozone levels.

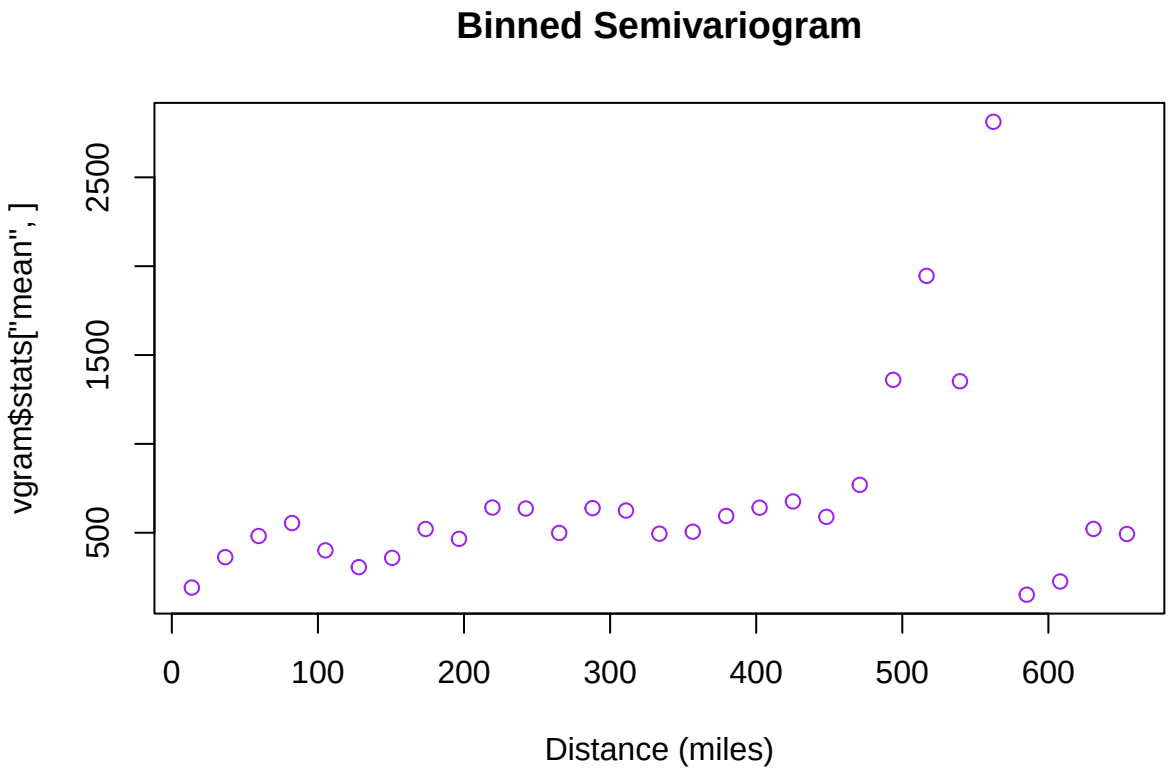
The plot shows clear spatial variation, with higher concentrations clustered in specific regions and lower concentrations in others. This pattern indicates spatial dependence, meaning that nearby locations tend to have similar ozone levels. The presence of this spatial structure suggests that ozone concentrations are not randomly distributed across space. Therefore, interpolation methods are appropriate for estimating ozone levels at unobserved locations by leveraging the spatial correlation in the data.

Variogram Analysis

```
lm_fit <- lm(y[good] ~ loc[good, ])

d <- rdist.earth(loc[good, ])
vgram <- vgram(
  loc = loc[good, ],
  y = lm_fit$residuals,
  N = 30,
  lon.lat = TRUE
)

plot(vgram$stats["mean", ] ~ vgram$centers,
     main = "Binned Semivariogram",
     xlab = "Distance (miles)",
     col = "purple")
```



A linear model was first fit using the spatial coordinates (longitude and latitude) to remove any large-scale spatial trend from the data. The residuals from this model represent the variation not explained by location and are used to analyze spatial dependence. Pairwise distances between observation locations were computed to measure how similarity changes with distance, and the empirical semivariogram was constructed by grouping observation pairs into distance bins and calculating the average squared difference within each bin.

The binned semivariogram shows how variability increases with distance. At shorter distances, the semivariance is relatively low, indicating that nearby locations have similar ozone levels. As distance increases,

the semivariance generally increases, suggesting that observations become less similar the farther apart they are. This pattern confirms the presence of spatial dependence in the data. The eventual leveling off of the semivariance indicates a range beyond which locations are no longer strongly correlated, highlighting the distance over which spatial influence persists.

Gaussian Process Model

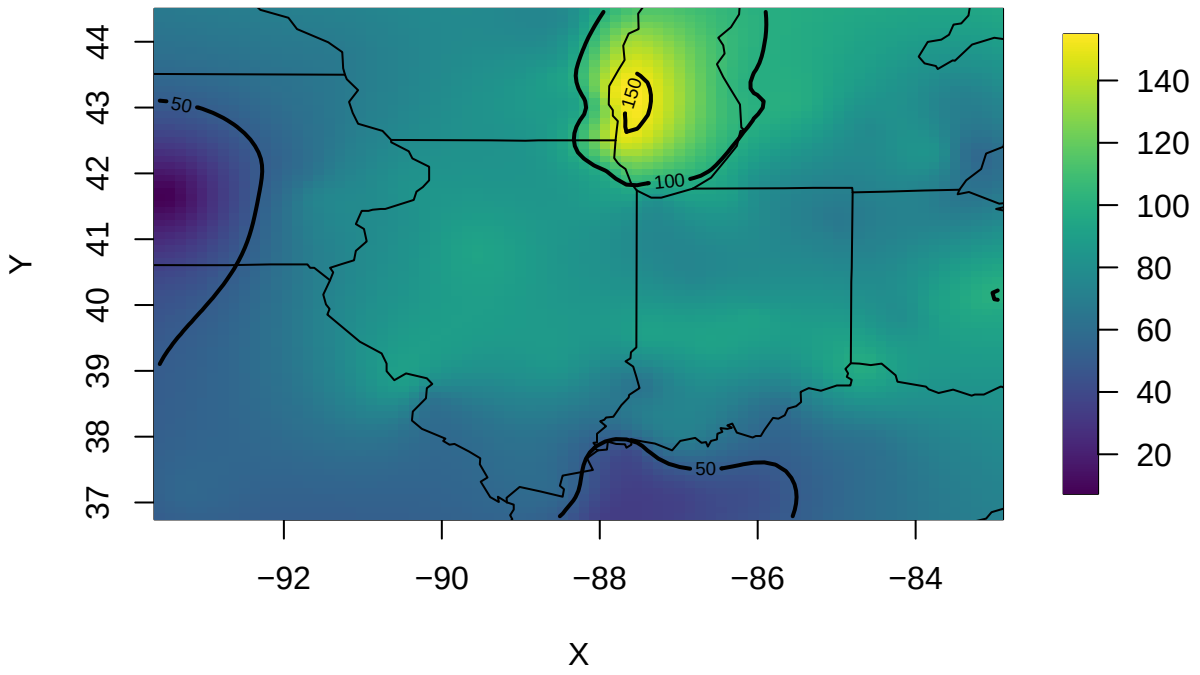
```
fit <- spatialProcess(loc[good, ], y[good])

out.full <- predictSurface(fit, extrap = TRUE)

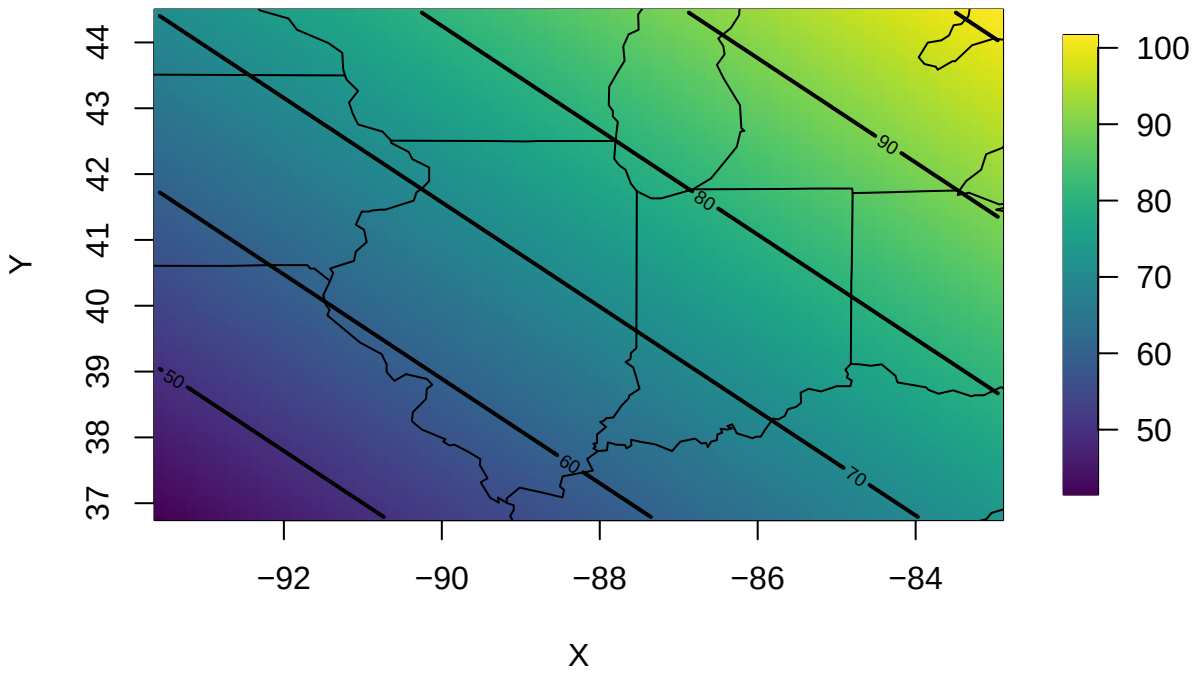
out.poly <- predictSurface(fit, just.fixed = TRUE, extrap = TRUE)

out.spatial <- out.full
out.spatial$z <- out.full$z - out.poly$z

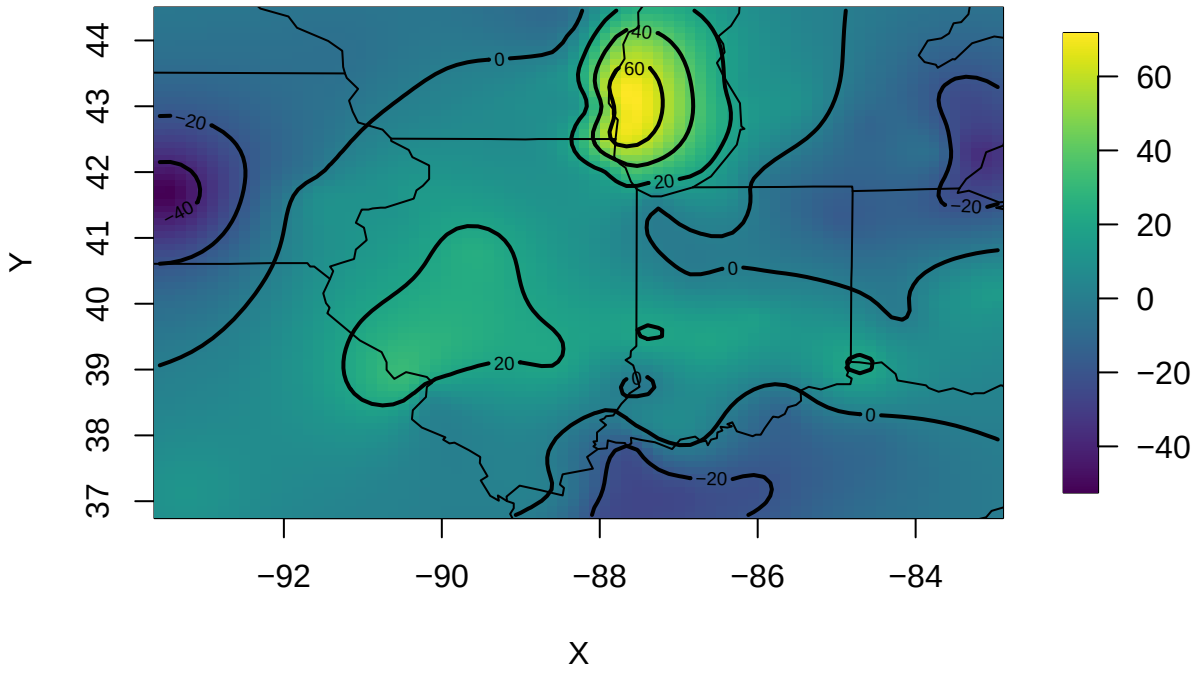
surface(out.full)
map("state", add = TRUE)
```




```
surface(out.poly)
map("state", add = TRUE)
```




```
surface(out.spatial)
map("state", add = TRUE)
```



A Gaussian process model was fit using the spatial coordinates and observed ozone values to capture both large-scale spatial trends and local spatial dependence. This approach models ozone levels as a smooth surface, where nearby locations are more strongly correlated than distant ones.

Predicted values were generated over a grid of locations to visualize the spatial distribution. The full prediction surface represents the combined effect of the large-scale trend and localized spatial variation. The trend surface isolates the broad spatial pattern, showing a gradual change in ozone levels across the region, while the residual (spatial) surface highlights localized deviations from this trend.

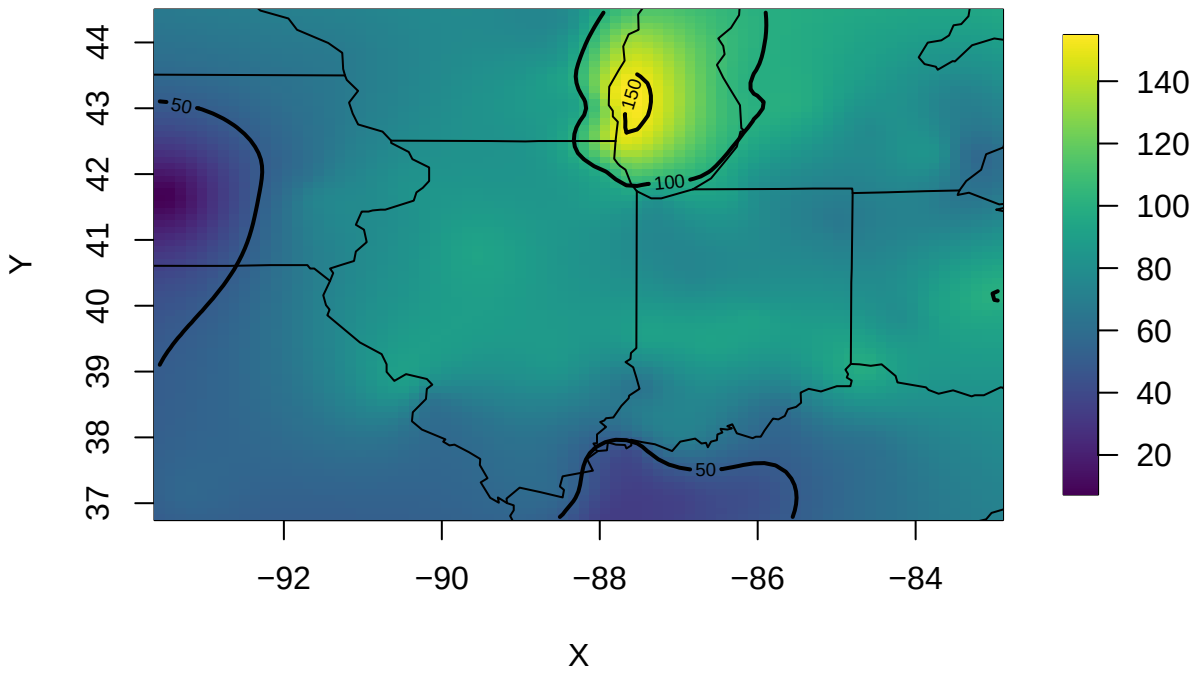
The full surface reveals clear spatial structure, with regions of higher ozone concentrations clustered in specific areas and lower concentrations elsewhere. The trend surface indicates a smooth gradient across the region, suggesting a systematic large-scale spatial pattern. In contrast, the residual surface captures smaller-scale variability, identifying localized areas where ozone levels are higher or lower than expected based on location alone.

Overall, these results demonstrate that ozone levels vary both smoothly across space and locally, confirming the presence of spatial dependence. The Gaussian process model effectively captures this structure, providing accurate spatial predictions and a meaningful decomposition of large-scale trends and localized variation.

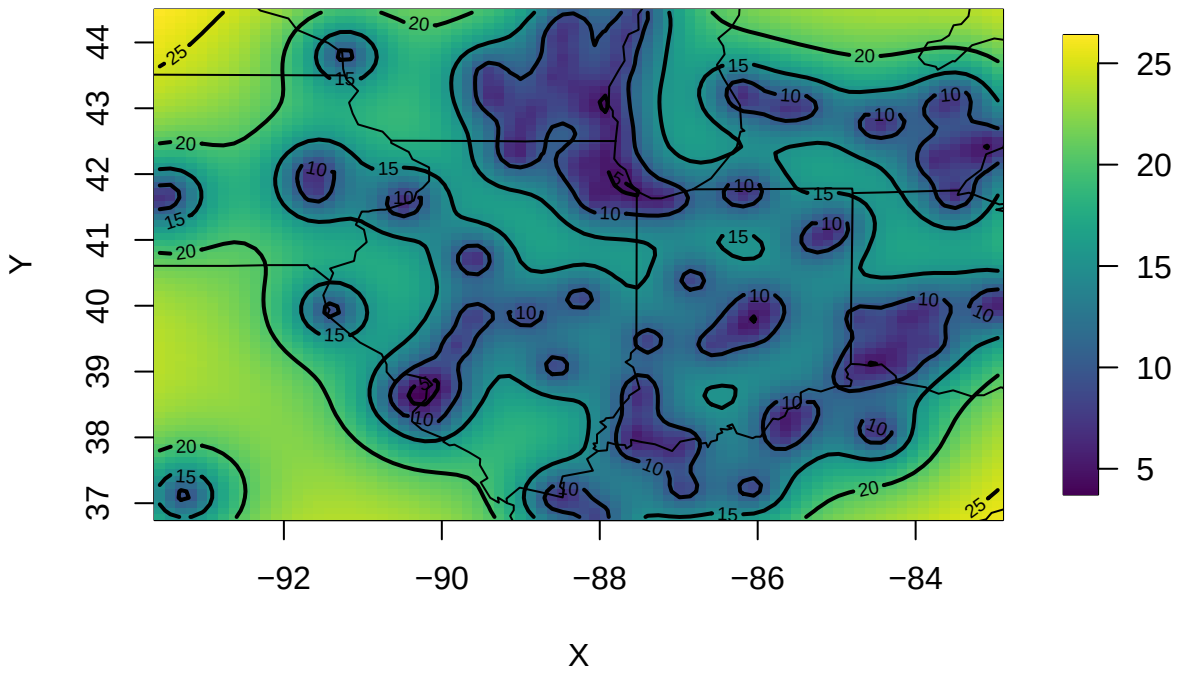
Prediction and Uncertainty

```
xg <- fields.x.to.grid(loc[good, ])
Pred <- predict.mKrig(fit, xnew = expand.grid(xg))
SE <- predictSE(fit, xnew = expand.grid(xg))

surface(as.surface(xg, Pred))
map("state", add = TRUE)
```




```
surface(as.surface(xg, SE))
map("state", add = TRUE)
```



The prediction map is generated using a Gaussian process model, which estimates ozone levels at unobserved locations by leveraging the spatial covariance structure of the data. This results in a smooth surface where nearby observations have a stronger influence on predictions than distant ones, reflecting spatial dependence.

The prediction surface shows a clear spatial pattern, with higher ozone concentrations concentrated in the upper-central region of the domain (approximately near longitude -88 and latitude 43) and lower values toward the western and southern areas. This indicates that ozone levels are not randomly distributed, but instead exhibit structured spatial variation driven by location.

The standard error surface quantifies the uncertainty associated with these predictions. Uncertainty is lowest in regions where observations are densely clustered, as the model has more information to rely on, and increases in areas with sparse data or near the boundaries of the domain. This reflects the reduced reliability of predictions where fewer nearby observations are available.

Overall, the results demonstrate that ozone levels exhibit strong spatial dependence, with both smooth large-scale patterns and localized variation. The Gaussian process model effectively captures this structure, providing reliable predictions while appropriately accounting for increased uncertainty in data-sparse regions.